UNIVERSIDAD RAFAEL NÚÑEZ

Especialización en Analítica de Datos · Tecnologías de Procesamiento de Datos

# Unidad 1: Fundamentos del Procesamiento Distribuido

*Sesión 1 · Viernes 30 de octubre · 5:00–9:00 p. m.*

# El problema: lo que necesita saber la gerencia

*Viajes de Uber y Lyft en Nueva York, junio de 2025 · datos abiertos de la Comisión de Taxis y Limusinas (TLC)*

| 19.868.009 | 4,04 GB | ≈ 662.000 | 71,3 % · 28,7 % |
|---|---|---|---|
| viajes en un mes | en CSV · 25 columnas por viaje | viajes por día | Uber · Lyft |

**Pregunta 1 · con el mes completo (por lotes)**

**¿Cuántos viajes hizo cada empresa, cuánto cobró de tarifa base, cuánto pagó a sus conductores y qué parte se quedó la plataforma?**

*Es la pregunta con la que Parrott y Reich (2018) sustentaron ante la TLC el pago mínimo para los conductores, vigente desde 2019.*

**Pregunta 2 · mientras ocurre (en tiempo real)**

**¿En qué zonas están esperando más los pasajeros y cuánto ingresa cada empresa, minuto a minuto?**

*Es la señal de una tarifa dinámica: si la espera sube en una zona, se sube el precio o se mueven conductores hacia allá.*

**El ejercicio de Tecnologías de Procesamiento de Datos: responder estas dos preguntas con el archivo real, completo y a tiempo. Ese es el problema; las tecnologías son la forma de resolverlo.**

# Por qué no se responde a tiempo, y cuánto cuesta

*El negocio: US$ 556,9 millones de tarifa base en junio · US$ 127,0 millones retenidos por las plataformas (22,8 %)*

| Causa 1 · Volumen | Causa 2 · Velocidad | Causa 3 · Calidad | Causa 4 · Capacidad fija |
|---|---|---|---|
| Excel carga 1.048.576 filas: ve el 5,3 % del mes | Unos 460 viajes por minuto; el reporte nocturno llega tarde | 231.332 esperas negativas y 12.901 tarifas en cero o menos | Un computador no crece con los datos; comprar para el pico deja equipos ociosos |

## US$ 12,2 M

en el mes, en 2.895.195 viajes (14,6 %) en los que el pago al conductor superó la tarifa base. ¿En qué zonas y a qué horas? Solo se sabe procesando el mes completo.

## US$ 49,9 M

de tarifa en 1.482.132 viajes (7,5 %) con más de 10 minutos de espera. Los que cancelaron no aparecen en el archivo: esa pérdida solo se ve en tiempo real.

## US$ 8,07 M

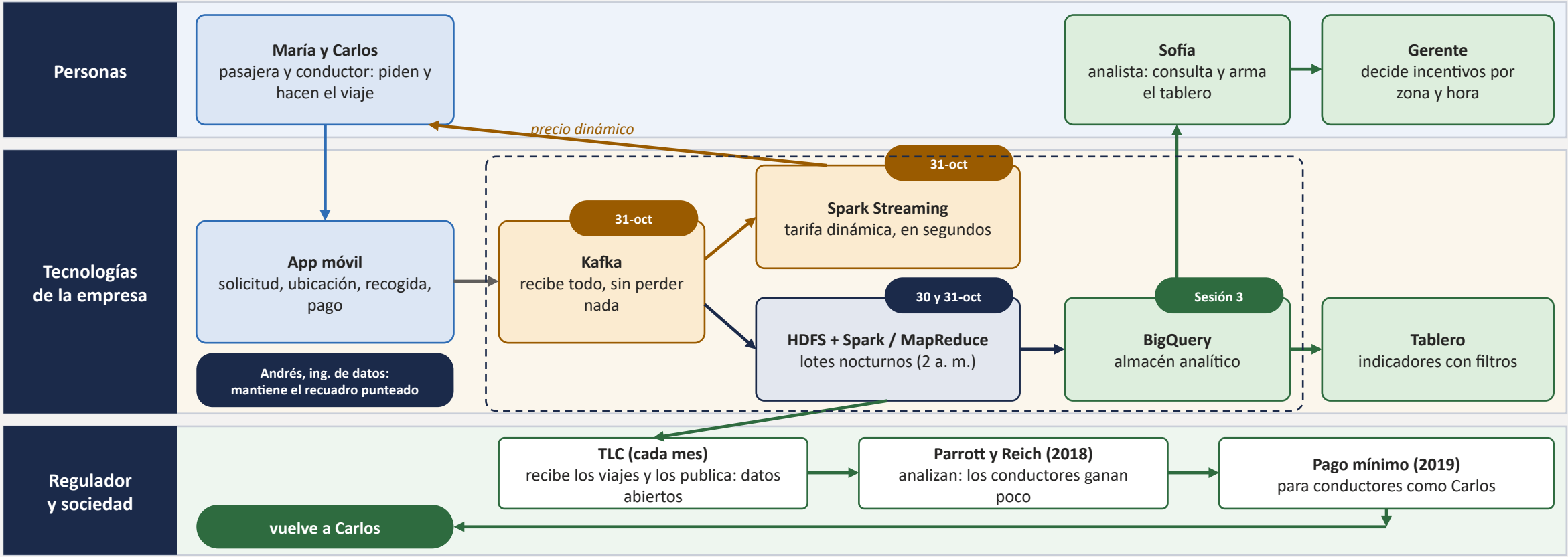
deja a las plataformas LaGuardia en un mes, la zona más rentable. Saber dónde se gana también es parte del problema.

*Cifras del archivo completo de la TLC. Plataforma = tarifa base menos pago al conductor (sin peajes, impuestos ni propinas). «En riesgo» no es «perdido»: estimar cuánto se recupera es trabajo de analista.*

**Cada causa tiene una tecnología que la ataca: eso es lo que vamos a medir en estas 4 sesiones.**

# Una noche en Nueva York: la cadena de la información

*Mapa de procesos del caso: quién genera cada dato, por dónde viaja y quién decide con él*




**En estas 4 sesiones seremos Andrés y Sofía: cada tecnología es un eslabón de esta cadena, y el archivo de la práctica es el que publica la TLC.**

*Personajes ilustrativos · tecnologías y hechos documentados: Fu y Soman (2021), TLC, Parrott y Reich (2018)*

# El objetivo del ejercicio: de la pregunta a la tecnología

*Cada herramienta entra cuando la anterior se queda corta. Así se articula toda la asignatura*

| Excel y pandas | Hadoop | Spark | Kafka + Streaming | La nube |
|---|---|---|---|---|
| un solo computador | HDFS + YARN + MapReduce | en memoria, con SQL | eventos en tiempo real | BigQuery + tablero |
| Excel no abre el mes; pandas pediría unos 11 GB de memoria | Reparte el archivo en 31 bloques y el cálculo en 31 tareas | El mismo resultado, más rápido; lee directo el Parquet oficial | Responde la pregunta 2 mientras ocurren los viajes | Consulta el mes sin servidores propios y lleva la respuesta a la gerencia |
| **Sáb 31 · Taller 1A · U1** | **Sáb 31 · Taller 1A · U1** | **Sáb 31 · Taller 1A · U1** | **Sáb 31 · Taller 1B · U3** | **Sesión 3 · Taller 2 · U2** |

**Lo que el estudiante hace en cada paso, como analista que consume estas plataformas:**

- **Mide:** tiempo, datos leídos, datos movidos y costo
- **Compara:** el mismo resultado con otra herramienta
- **Decide:** qué tecnología responde cada pregunta y por qué

**Objetivo: decidir con evidencia qué tecnología de procesamiento responde cada necesidad del negocio, completa, exacta y a tiempo.**

# ¿Por qué hacerlo en la nube?

*El analista no compra servidores: pide capacidad cuando la necesita y la devuelve cuando termina*

## Con el propio computador

- Hay que instalar y mantener todo: Docker, Hadoop, Spark, Kafka
- Cada equipo es distinto: lo que funciona en uno falla en otro
- La capacidad es fija: 8 o 16 GB no alcanzan cuando los datos crecen (un año del caso son unos 48 GB en CSV)
- Si el equipo falla o se apaga, se detiene el trabajo

## En la nube

- Elasticidad: se piden 4 núcleos hoy y 400 mañana, por horas
- Pago por uso: se paga lo que se procesa (BigQuery cobra por los datos que lee cada consulta)
- Servicios administrados: el proveedor instala y opera; el analista consulta
- Desde el navegador: el mismo entorno para todos, en cualquier equipo

**En esta asignatura:**

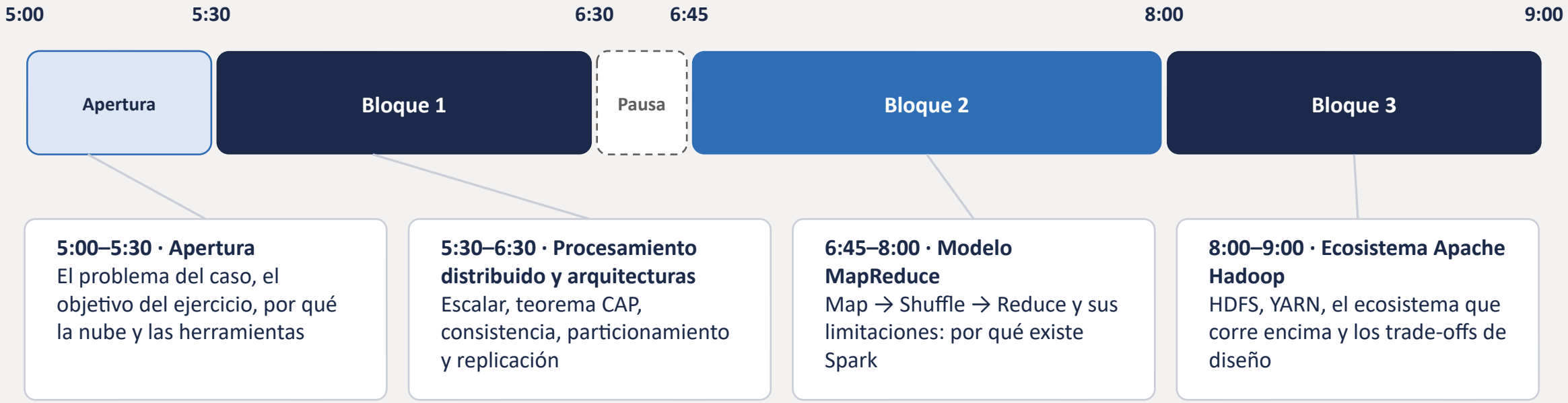
**Sesiones 1 y 2 · GitHub Codespaces:** un computador en la nube de 4 núcleos y 16 GB, gratis y en el navegador, con Hadoop, Spark y Kafka

**Sesiones 3 y 4 · Google Cloud:** BigQuery consulta los 19,9 millones de viajes sin servidores propios y sin tarjeta (modo Sandbox)

**La nube no siempre es la respuesta:** datos sensibles con restricciones legales, o cargas constantes que salen más caras alquiladas que propias. Ese análisis (seguridad, normativa y costos) es la Unidad 2.

# Agenda de hoy

*Hoy, la Unidad 1 — mañana sábado 31, la práctica en GitHub Codespaces*




**Hoy, el marco para evaluar arquitecturas. Mañana, sábado 31-oct, en GitHub Codespaces: Hadoop y Spark (8:00 a. m.) y Kafka con la app de viajes (tarde).**

# Las herramientas de Tecnologías de Procesamiento de Datos

*Qué usaremos, para qué y cuándo. Detalle: Preparación Previa (GitHub Codespaces)*

## Entorno de trabajo

Todas las sesiones

### GitHub Codespaces

Computador en la nube (4 núcleos, 16 GB) desde el navegador

### Docker

Ya viene en el codespace: Hadoop, Spark y Kafka en contenedores

### Terminal (bash)

Para dar las órdenes, en la misma pestaña

### Python

Preparar los datos; app de viajes y clientes de Kafka

## Unidad 1 · Lotes

Sáb 31-oct, mañana

### Hadoop: HDFS + YARN

Datos repartidos en bloques y recursos del clúster

### MapReduce

Procesamiento por lotes en dos fases

### Spark

El mismo cálculo en memoria, para comparar

## Unidad 3 · Tiempo real

Sáb 31-oct, tarde

### App de viajes (Streamlit)

Cliente, conductor y central publican cada evento del viaje

### Kafka

Mueve eventos en tiempo real entre sistemas

### Spark Structured Streaming

Procesa esos eventos en ventanas de tiempo

## Unidad 2 · Nube

Sesiones 3 y 4

### Spark en Google Cloud

Dataproc: demostración del docente

### BigQuery

SQL sin servidores; costo por bytes procesados

### Looker Studio

Tablero de indicadores para la gerencia

### AWS y Azure

Comparativa de servicios; práctica opcional

**También existen (solo para ubicarse, no se practican):** Podman · Kubernetes · Cloudera · Amazon EMR · Azure HDInsight · Apache Flink · Databricks · Amazon Redshift · Azure Synapse · Snowflake · Apache Pulsar · Amazon Kinesis · Google Pub/Sub

# Por qué estas herramientas

*Criterios de elección. Lo que se aprende con ellas sirve con cualquier proveedor*

1. **Lo pide el contenido programático**
2. **Sin costo: cuota gratuita de GitHub y BigQuery Sandbox, sin tarjeta**
3. **Corre igual para todos: en el navegador, sin instalar**
4. **Lo aprendido sirve con las alternativas**

| Categoría | Usamos | Alternativa principal | Por qué la nuestra |
|---|---|---|---|
| Entorno | **GitHub Codespaces** | Docker Desktop en el PC | 4 núcleos y 16 GB en el navegador; nadie instala nada |
| Contenedores | **Docker (en el codespace)** | Podman | Imágenes oficiales de Hadoop, Spark y Kafka |
| Terminal | **bash** | PowerShell | La terminal de los servidores Linux donde corren estas plataformas |
| Preparar datos | **Python + pandas** | Excel / Power Query | El dataset supera el máximo de filas de una hoja |
| Lotes distribuidos | **Apache Hadoop** | Amazon EMR | Gratis y muestra HDFS, YARN y MapReduce por separado |
| Motor en memoria | **Apache Spark** | Apache Flink | Un solo motor para lotes (U1) y tiempo real (U3) |
| Nube | **Google BigQuery** | Amazon Redshift | Muestra el costo de cada consulta; Sandbox sin tarjeta de respaldo |
| Tiempo real | **Apache Kafka** | Amazon Kinesis | Gratis; sus conceptos se usan en los servicios de nube |

# Cómo vamos a trabajar: qué sí, qué no y por qué

*En la empresa, el analista no construye la plataforma: la usa, la mide y decide con ella*

| Sí haremos | No haremos | Por qué |
|---|---|---|
| Ejecutar con el mes completo los procesos ya preparados | Instalar Hadoop, Spark o Kafka, ni montar servidores | En la empresa eso lo hacen ingeniería de datos o el proveedor de nube; el codespace ya lo trae listo |
| Configurar: cambiar memoria, núcleos o particiones y medir el efecto (RA1, RA4) | Configurar un clúster desde cero | Configurar, aquí, es ajustar un sistema que ya funciona y demostrar con mediciones que mejoró |
| Poner a andar el flujo en tiempo real con la app de viajes y comprobar que no pierde eventos (RA3) | Programar productores y consumidores desde cero | El analista necesita saber si el flujo es confiable y exacto, no escribir su código |
| Leer el código y explicar qué hace cada parte | Escribir o depurar programas | Basta entender cómo trabaja la herramienta para saber qué pedirle y qué esperar de ella |
| Consultar con SQL y armar el tablero (Spark, BigQuery, Looker Studio o Power BI) | Programar en Java o Scala | SQL y el tablero son las herramientas diarias del analista |
| Comparar tiempo, costo y exactitud, y recomendar una tecnología (RA2) | Memorizar comandos | Eso es lo que se evalúa: la decisión justificada con sus propias cifras |

*¿Y el tiempo? Las horas son las mismas: la máquina tarda lo mismo con o sin programar (cada job de Hadoop con el mes, 2,5 a 4,5 min). Lo que no se pierde corrigiendo código se usa en interpretar y decidir. Cada comando se copia de la guía, que muestra la salida esperada y qué hacer si algo falla.*

**Su papel es el de Sofía, la analista: lo que se evalúa es la decisión que toman con los datos, no escribir código.**

Bloque 1

# Procesamiento distribuido y arquitecturas

5:30–6:30 p. m.

> *¿Qué pasa cuando una base de datos relacional ya no cabe en una sola máquina?*

**Bloque 1 · Procesamiento distribuido**

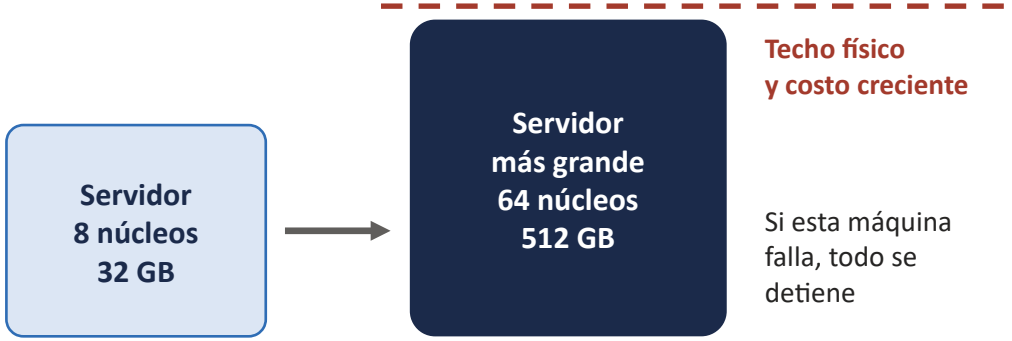
Bloque 2 · Modelo MapReduce

Bloque 3 · Ecosistema Hadoop

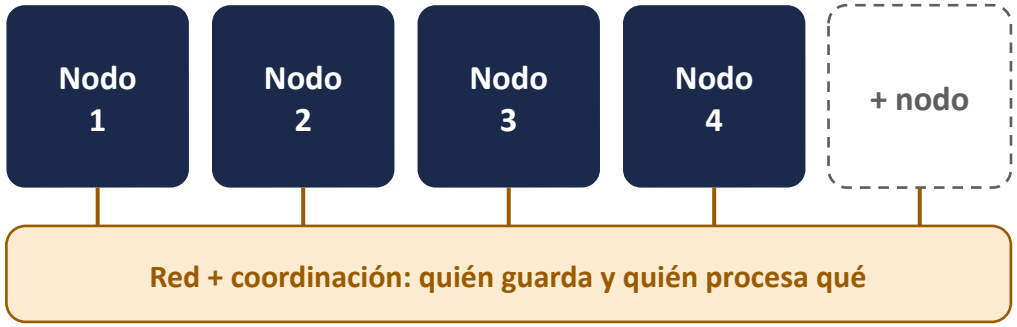
# ¿Qué pasa cuando una sola máquina no alcanza?

*Cuando los datos crecen hay dos caminos: una máquina más grande o más máquinas trabajando juntas*

### Escalar verticalmente




### Escalar horizontalmente




*Crece agregando nodos comunes; si uno falla, los demás siguen.*

- **Límite físico:** CPU, memoria y disco de una sola máquina son finitos.
- **Escalar verticalmente:** más potencia a la misma máquina. Simple, pero tiene techo y costo creciente.
- **Escalar horizontalmente:** más máquinas trabajando juntas. Sigue creciendo, pero exige coordinación.

### Escenario: el cierre nocturno de viajes deja de caber

Cada noche se consolidan los millones de viajes del día. El volumen crece y el proceso ya no termina antes del amanecer.

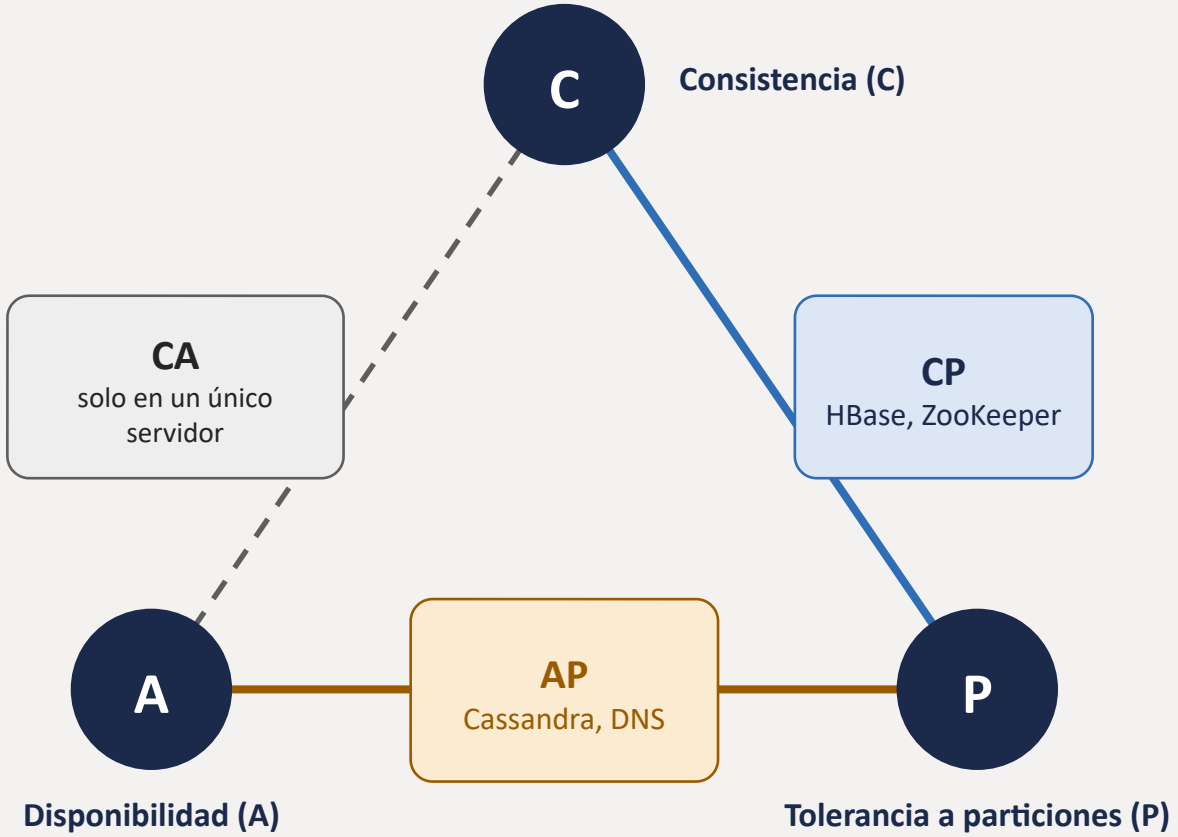
**Camino vertical**
Un servidor más grande: alcanza hasta el próximo techo y sigue siendo un único punto de falla.

**Camino horizontal**
Repartir los viajes entre varios nodos: se crece agregando nodos, pero hay que coordinarlos.

**Escalar horizontalmente exige coordinación: esa coordinación es exactamente lo que estudia el procesamiento distribuido.**

# Teorema CAP

*Si la red se parte, un sistema distribuido debe elegir: consistencia o disponibilidad*




- **C:** toda lectura devuelve la escritura más reciente, sin importar qué nodo responda.
- **A:** todo nodo que funciona responde siempre, aunque otros nodos fallen.
- **P:** el sistema sigue operando aunque se pierdan mensajes entre nodos.

### Escenario: la red se parte y alguien consulta un saldo

El nodo 1 registró un retiro; el nodo 2 no alcanzó a recibirlo. Un cliente consulta el saldo en el nodo 2.

**Sistema CP**
Responde «no disponible» o espera. Nunca entrega un saldo desactualizado.

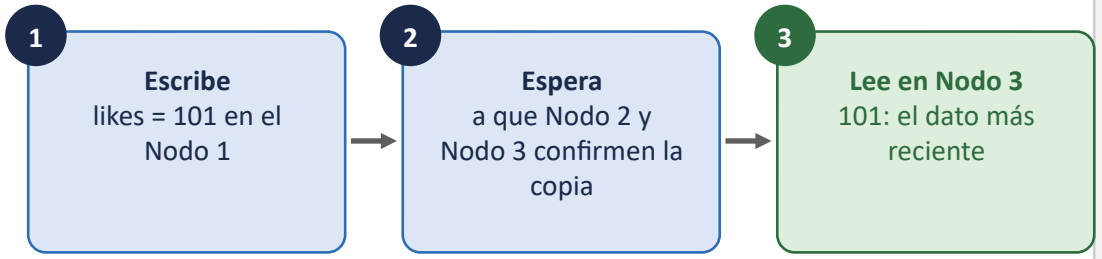
**Sistema AP**
Responde de inmediato, pero con el saldo anterior al retiro.

**En un sistema distribuido real las particiones ocurren: P no se elige. La decisión es entre C y A, y es de negocio.**

# Consistencia fuerte vs. consistencia eventual

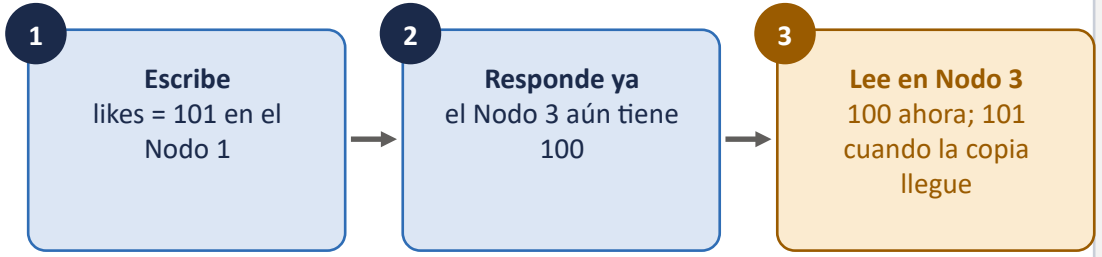
*Qué ve quien lee justo después de una escritura, cuando el dato tiene tres réplicas*

### Consistencia fuerte




*La escritura no termina hasta que las réplicas confirman: más espera, siempre correcta.*

### Consistencia eventual




*Responde sin esperar: más disponible; las réplicas convergen después.*

- **Consistencia fuerte:** toda lectura devuelve el dato más reciente. Más simple de razonar, más costosa de coordinar.
- **Consistencia eventual:** las réplicas convergen con el tiempo. Más disponible y escalable; exige diseñar la aplicación para tolerarlo.

### Escenario: el mismo retraso, dos costos de negocio

Una réplica va un instante atrasada. ¿Cuánto cuesta que alguien lea el valor anterior?

**Contador de «likes»**
Ver 100 en vez de 101 por un instante no le cuesta nada a nadie: eventual basta.

**Saldo bancario**
Leer un saldo anterior al retiro permite gastar dos veces el mismo dinero: exige fuerte.

**La elección no es solo técnica: depende del costo de negocio de un dato desactualizado.**

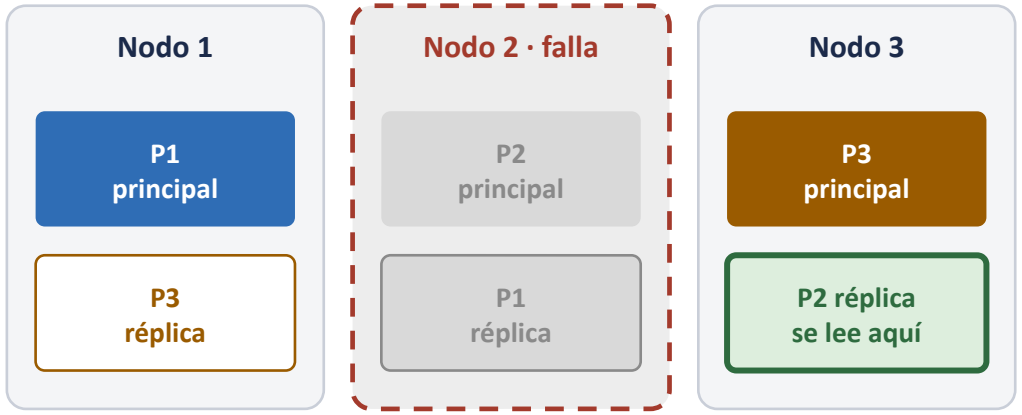
# Tolerancia a fallos y particionamiento

*Particionar reparte la carga; replicar permite sobrevivir a la falla de un nodo*

### 1. Particionar los viajes por distrito




### 2. Guardar cada partición en un nodo y su réplica en otro




**Cae el Nodo 2:** P2 se sigue leyendo desde su réplica en el Nodo 3, y P1 nunca dependió de él. Sin réplicas, los viajes de Brooklyn quedarían inaccesibles.

- **Particionamiento (sharding):** dividir los datos entre nodos para distribuir la carga.
- **Replicación:** mantener copias del mismo dato en varios nodos para sobrevivir a fallos.

### Escenario: un nodo se apaga en plena consulta nocturna

Los viajes del caso están repartidos por distrito de recogida en 3 nodos, como en el diagrama.

**Solo particionado**
Los viajes del distrito de ese nodo quedan inaccesibles hasta que el nodo vuelva.

**Particionado y replicado**
La consulta sigue; el costo es guardar cada dato más de una vez y mantener las copias al día.

**Todo diseño distribuido combina ambos, y cada combinación implica un trade-off distinto de costo, velocidad y riesgo.**

Bloque 2

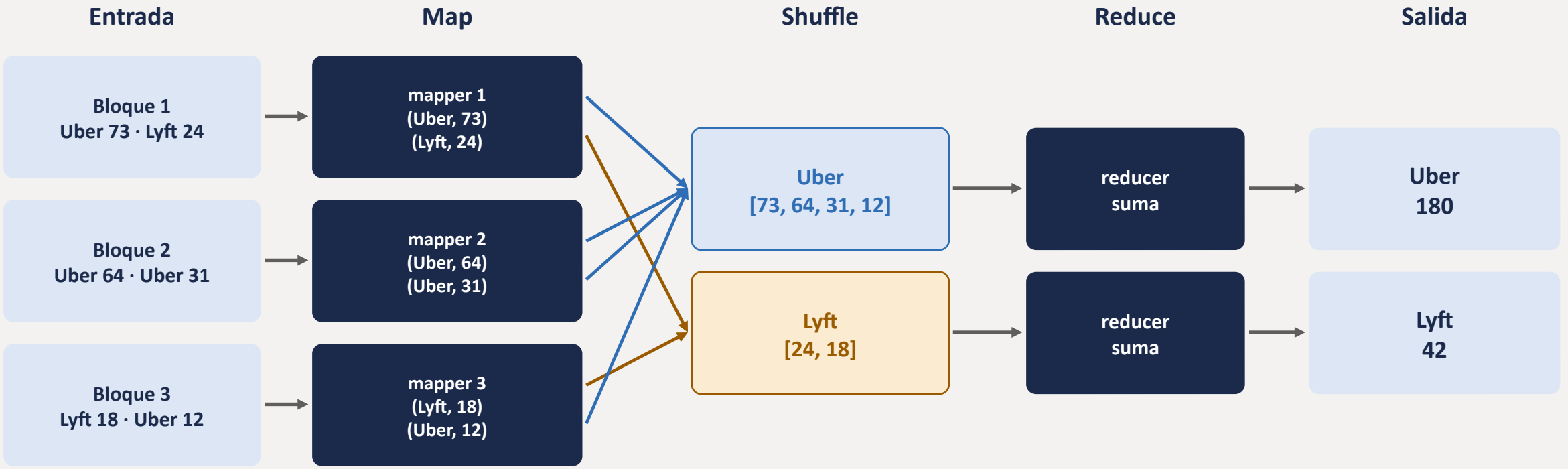
# Modelo MapReduce

6:45–8:00 p. m.

> *¿Qué pasos de un problema de su trabajo serían «map» y cuáles «reduce»?*

# Map → Shuffle → Reduce

*El job del taller de mañana: tarifa base por empresa*




*Tarifas ilustrativas, en dólares. Las flechas de color son el shuffle: cada par viaja por la red al reducer de su clave.*

**Map**
cada nodo procesa su porción de datos de forma independiente y en paralelo.

**Shuffle**
los resultados intermedios se agrupan por clave y se redistribuyen entre nodos.

**Reduce**
cada nodo agrega los valores de una clave y produce el resultado final.

**El shuffle es la fase más costosa: mueve datos por la red entre todos los nodos.**

# Actividad: MapReduce a mano

*15 minutos · 12 viajes reales del archivo de la TLC · grupos de estudiantes hacen de mappers y de reducers*

| | | | | |
|---|---|---|---|---|
| **Bloque 1 mapper 1** | Uber US$ 73,25 | Uber US$ 64,49 | Lyft US$ 22,44 | Uber US$ 32,99 |
| **Bloque 2 mapper 2** | Uber US$ 57,12 | Uber US$ 19,54 | Lyft US$ 17,90 | Uber US$ 8,84 |
| **Bloque 3 mapper 3** | Uber US$ 8,38 | Uber US$ 28,00 | Lyft US$ 6,40 | Lyft US$ 12,71 |

*Tarifa base de cada viaje, en dólares. Una tarjeta por viaje.*

**1. Map:** Cada grupo mapper escribe un papel por viaje: (empresa, tarifa).

**2. Shuffle:** Los papeles viajan: todos los de Uber al grupo reducer Uber y los de Lyft al grupo Lyft. Cuenten los papeles que viajaron.

**3. Reduce:** Cada reducer suma: Uber US$ 292,61 · Lyft US$ 59,45.

**4. Combiner:** Repitan sumando primero dentro de cada mapper: ¿cuántos papeles viajan ahora?

**Sin combiner viajan 12 papeles; con combiner, 6 (dos por mapper). Con el mes completo: 19,9 millones de pares frente a 62.**

# Las limitaciones de MapReduce, no solo sus fortalezas

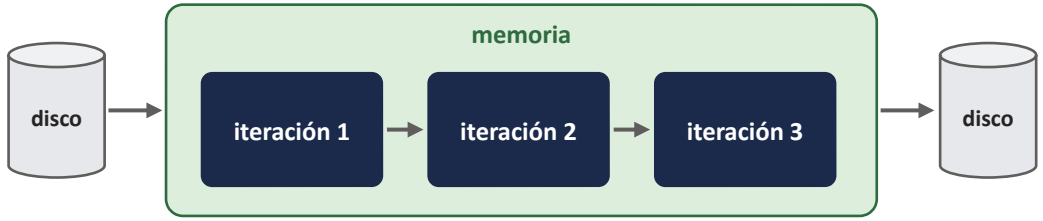
*Un algoritmo iterativo repite el mismo cálculo varias veces sobre los mismos datos*

### MapReduce: cada iteración es un job nuevo




**Cada iteración vuelve a leer del disco** lo que escribió la anterior.

### Spark: los datos quedan en memoria entre pasos




**Lee una vez, itera en memoria** y escribe el resultado al final.

- **Disco entre jobs:** cada job escribe los resultados intermedios a disco; costoso si el mismo dato se procesa varias veces.
- **Iterar es relanzar:** los algoritmos iterativos (p. ej., machine learning iterativo) relanzan el job en cada iteración.
- **Sin memoria compartida:** no hay forma nativa de mantener datos en memoria entre pasos de un mismo flujo.

### Escenario: un modelo que se ajusta en 10 pasadas

El mismo cálculo recorre los viajes 10 veces, refinando el resultado en cada pasada.

**Con MapReduce**
10 jobs: cada uno arranca, lee del disco lo que dejó el anterior y vuelve a escribir.

**Con Spark**
Los viajes se leen una vez y las 10 pasadas trabajan sobre los datos en memoria.

**Esta limitación concreta es la razón de ingeniería detrás de Apache Spark. Mañana la comprueban con métricas.**

Bloque 3

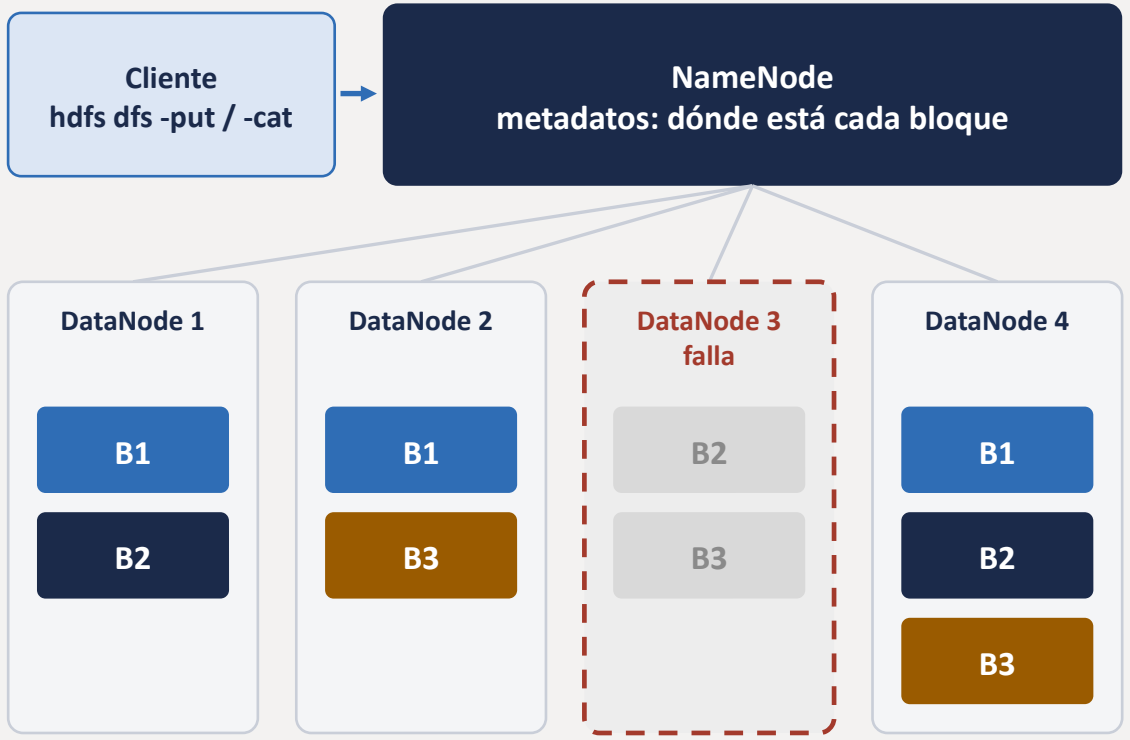
# Ecosistema Apache Hadoop

8:00–9:00 p. m.

> *¿Por qué HDFS replica cada bloque 3 veces por defecto?*

# HDFS: Hadoop Distributed File System

*Un archivo se divide en bloques (B1, B2, B3) y cada bloque se guarda 3 veces en máquinas distintas*




**Cae el DataNode 3:** B2 sigue en los DataNodes 1 y 4, y B3 en los DataNodes 2 y 4. Ningún dato se pierde.

- **NameNode:** mantiene los metadatos (qué bloque está en qué máquina); es el punto único de coordinación.
- **DataNodes:** almacenan los bloques reales de datos.
- **Réplicas:** cada bloque se replica 3 veces por defecto entre distintos DataNodes.

### Escenario: se apaga una máquina del clúster

¿Qué pasa si la que se apaga es un DataNode? ¿Y si es el NameNode?

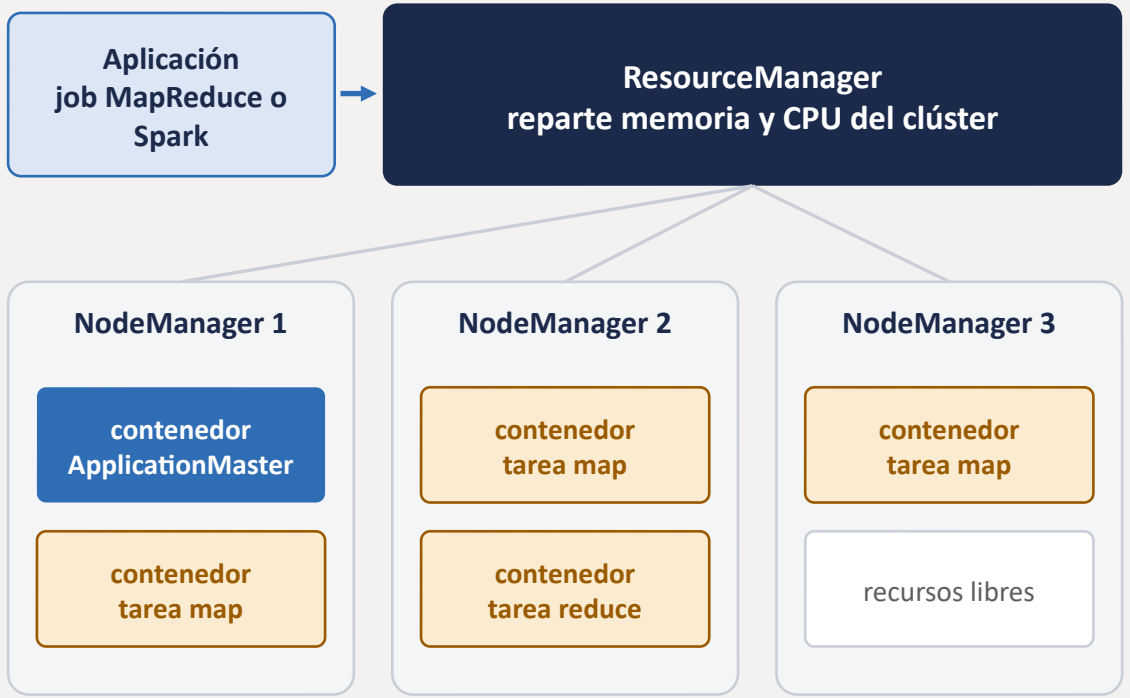
**Cae un DataNode**
Se leen las otras réplicas y el NameNode ordena recopiar los bloques hasta volver a 3.

**Cae el NameNode**
Los bloques siguen en disco, pero nadie sabe dónde está cada uno: HDFS se detiene.

**El NameNode único es una decisión de diseño con una consecuencia clara: sin alta disponibilidad, es un punto único de falla.**

# YARN: gestión de recursos y scheduling

*YARN reparte memoria y CPU del clúster en contenedores; cada aplicación coordina sus propias tareas*




1. La aplicación se envía al ResourceManager.
2. El ResourceManager lanza el ApplicationMaster de esa aplicación en un contenedor.
3. El ApplicationMaster pide contenedores y lanza en ellos las tareas map y reduce.

- **Separación:** YARN gestiona los recursos; el modelo de procesamiento (MapReduce, Spark, etc.) va aparte.
- **ResourceManager:** decide qué aplicación recibe qué recursos del clúster.
- **NodeManager:** supervisa el uso de recursos en cada nodo individual.
- **ApplicationMaster:** uno por aplicación; pide contenedores y coordina sus tareas.

### Escenario: dos motores en el mismo clúster

Un job MapReduce nocturno y un análisis con Spark llegan a la vez; YARN reparte el clúster entre ambos.

**MapReduce**
Su ApplicationMaster pide contenedores para map y reduce.

**Spark**
Su ApplicationMaster pide contenedores para sus ejecutores.

**Esta separación es lo que permitió que Spark corriera sobre la misma infraestructura que Hadoop.**

# El ecosistema sobre HDFS y YARN

*Informativo: herramientas que se apoyan en las capas vistas hoy (no se practican en el taller)*



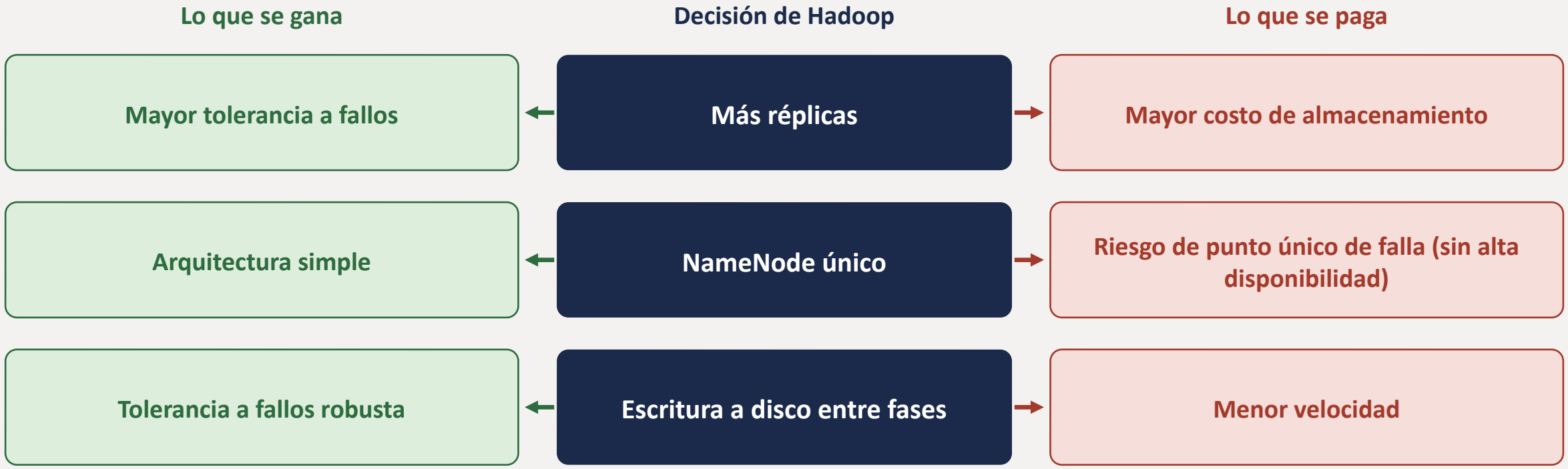

- **Hive:** tablas y SQL sobre archivos de HDFS. Pensado para lotes: cada consulta se vuelve un trabajo distribuido.
- **Impala:** SQL de baja latencia; ejecuta en memoria con sus propios procesos, sin pasar por MapReduce.
- **MLlib:** algoritmos de machine learning distribuidos en Spark. Los algoritmos iterativos aprovechan la memoria, justo la limitación de MapReduce que vimos hoy.

*Spark también puede correr sin YARN (modo local o standalone): así lo usarán mañana, en su propio contenedor.*

**Idea para llevarse:** el mismo HDFS sirve a motores distintos; elegir motor es elegir entre latencia, tipo de trabajo (lotes, interactivo, iterativo) y costo.

# Trade-offs de diseño: no hay arquitectura gratuita

*Cada decisión de Hadoop compra algo y lo paga con otra cosa*




**Cada decisión de Hadoop es defendible, y cada una tiene un costo que hay que poder justificar.**

# Cierre del día

## Hoy

Escalar | Teorema CAP | Consistencia | Particionamiento y replicación | MapReduce | HDFS | YARN | Trade-offs

## Mañana (sábado 31-oct, 8:00 a. m.)

1. **Abrir el codespace (Preparación Previa hecha)**
2. **Ejecutar y perfilar un job MapReduce real**
3. **Migrar a Spark y comparar**
4. **Arranca el Proyecto Final; en la tarde, Kafka y la app de viajes**

> *Traer la Preparación Previa hecha: el verificador debe decir «LISTO para el Taller 1». A las 8:00 a. m. no se instala nada.*